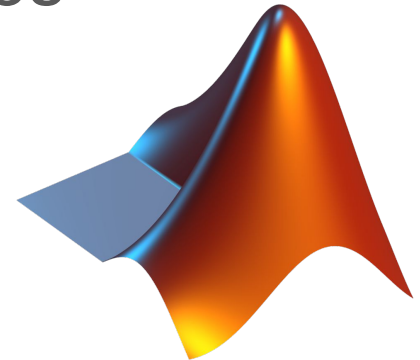


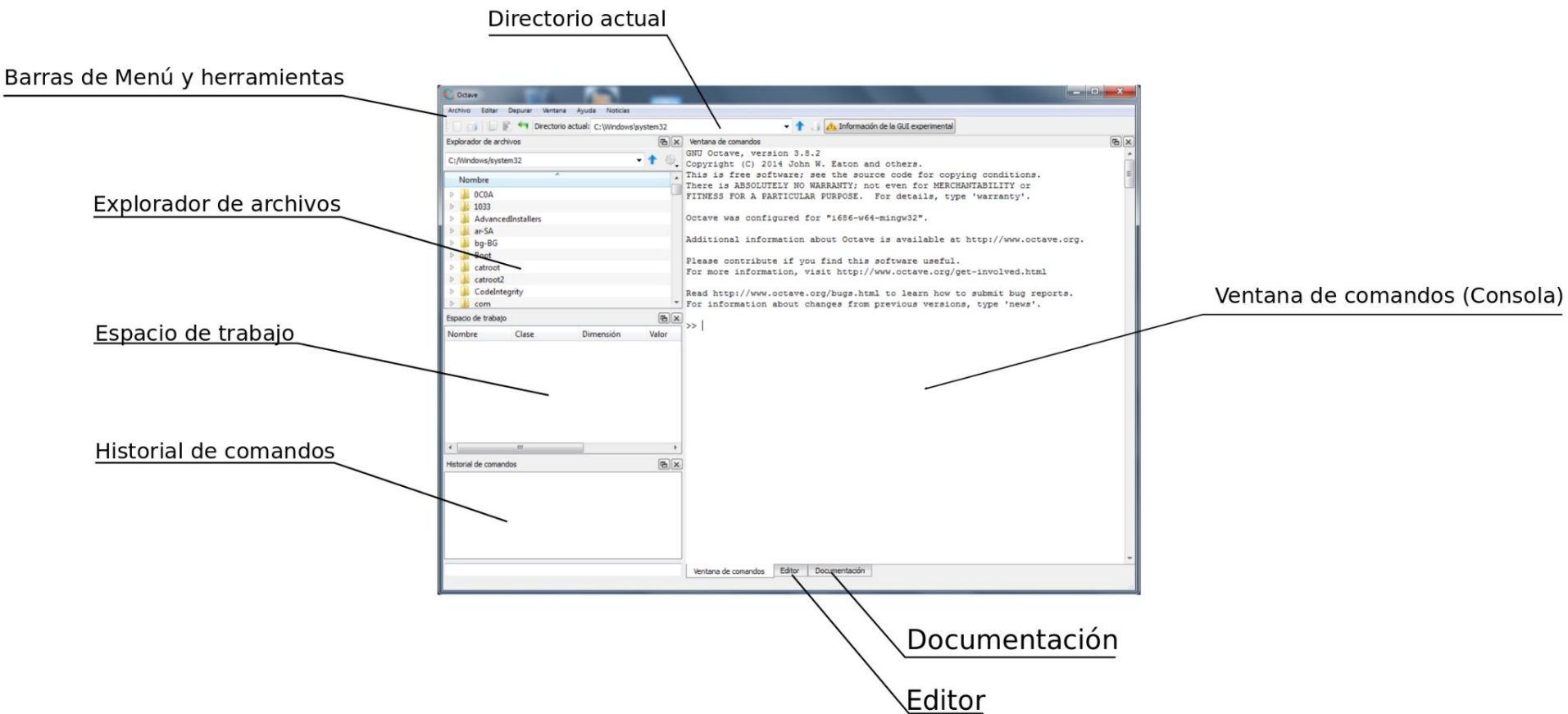


Introducción a MATLAB/OCTAVE/SCILAB

Complementos



Interfaz gráfica de usuario (GUI)



Operaciones

+	adición o suma
-	sustracción o resta
*	multiplicación
/	división
^	potenciación
\	división inversa

&	y (and)
	o (or) no excluyente
xor	o excluyente
~	negación lógica

<	menor que
>	mayor que
<=	menor o igual que
>=	mayor o igual que
==	igual que
~=	distinto que

- Podemos definir un **vector fila x** separando sus componentes por espacios en blancos o bien por comas (,):

```
>> x = [1 2 4]    (o bien x = [1, 2, 4])  
x =  
1    2    4      (vector fila)
```

- Si los números están separados por punto y coma (;) obtenemos un **vector columna y**:

```
>> y = [1; 3; 9]  
y =  
1  
3      (vector columna)  
9
```

- Definimos las **matrices** por filas separando los elementos de una misma fila por espacios en blancos o comas (,) y separando las filas por punto y coma (;):

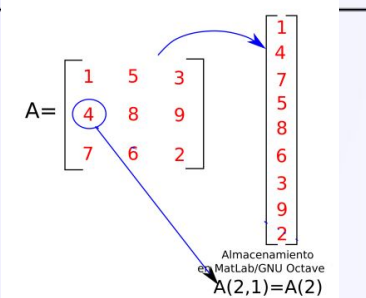
```
>> A=[8 1 6; 3 5 7; 4 9 2]
```

```
A =
```

```
8   1   6
```

```
3   5   7
```

```
4   9   2
```



- Podemos definir el **producto escalar interno** entre vectores del mismo espacio vectorial:

producto escalar externo (genera una matriz).

- Para multiplicar o dividir vectores o matrices *componente a componente*, utilizamos el **operador punto** (**.***) y (**./**):

Función predefinida y definidas por el usuario

log(x): logaritmo natural.

log10(x): logaritmo decimal.

exp(x): función exponencial.

abs(x) valor absoluto.

sqrt(x): raíz cuadrada.

- en archivos `.m` (*m-file functions*) o `.sci` (*Scilab*).
- en líneas de comandos (*inline, function_handle, deff*).



Editor - /media/luciano/Lucho/2015 - Matematica Aplicada/Laboratorios/Clase 1 - Intro + MATLAB/nom_func.m

File Edit Text Go Cell Tools Debug Desktop Window Help

Stack: Base fx

```

1 function [ argumentos_salida ] = nom_func( argumentos_entrada )
2 % nom_func: detalle de lo que hace la función
3 % INPUT: descripcion de los argumentos de entrada
4 % OUTPUT: descripcion de los argumentos de salida
5 % Autor/es y modificaciones: (recomendable)
6 % Funciones que usa: (recomendable)
7
8 argumentos_salida = argumentos_entrada; % -----> contenido de la función
9
10 end

```

```
arg_func = inline('expresión','arg_1','arg_2',...)
```

nom_func = @(arg_1,arg_2,...) expresión

Script

Crear un script llamado **matrices.m** que haga la adición, substracción y multiplicación de las siguientes matrices:

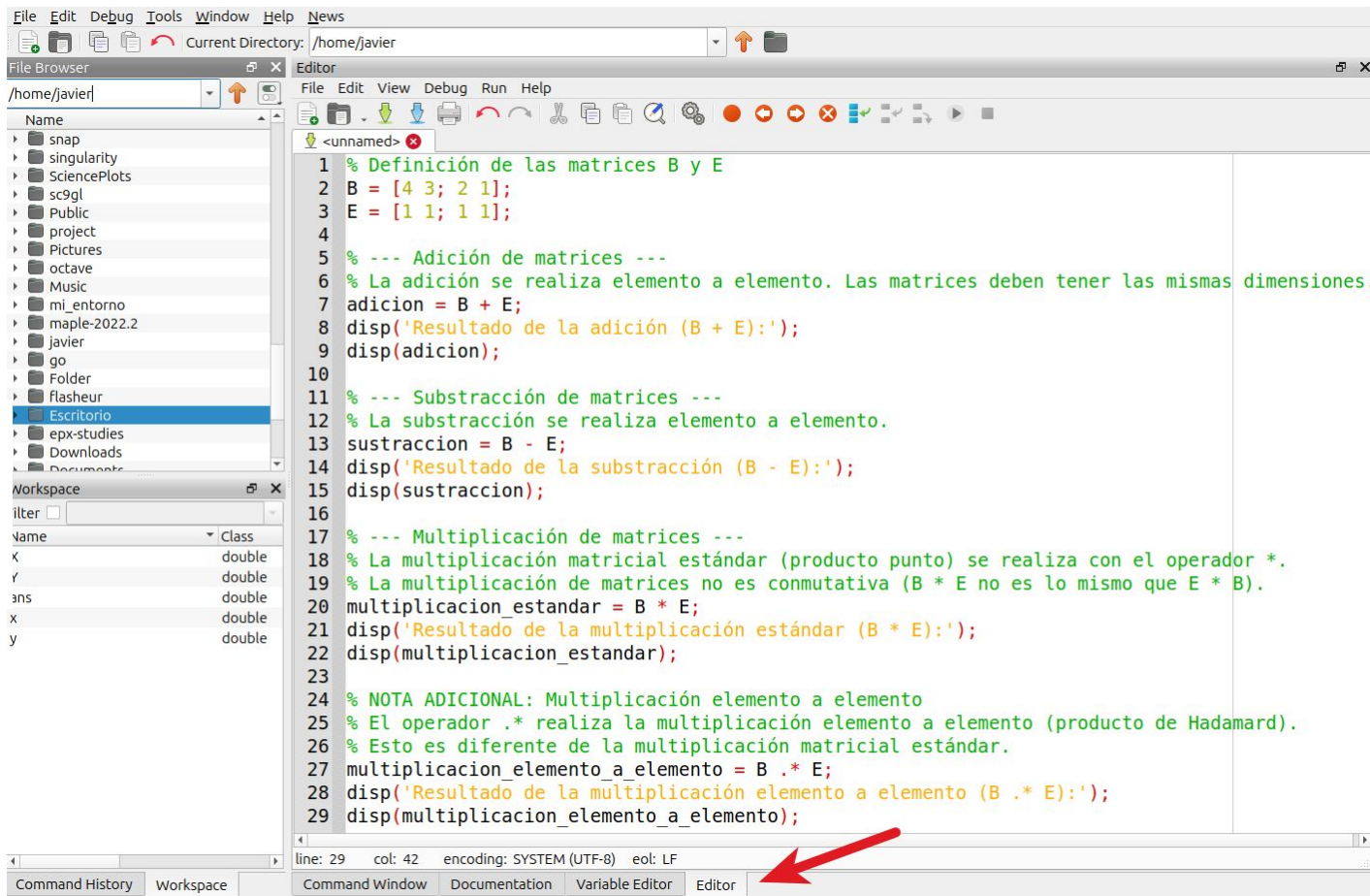
$$B = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} \quad E = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Utiliza los operadores aritméticos matriciales

Script

$$B = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$$

$$E = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$



```
File Edit Debug Tools Window Help News
Current Directory: /home/javier

File Browser
/home/javier/
Name
  snap
  singularity
  SciencePlots
  sc9gl
  Public
  project
  Pictures
  octave
  Music
  mi_entorno
  maple-2022.2
  javier
  go
  Folder
  flasheur
  Escritorio
  epix-studies
  Downloads
  Documents
Workspace
filter
Name Class
x double
y double
ans double
x double
y double

Editor
File Edit View Debug Run Help
<unnamed>
1 % Definición de las matrices B y E
2 B = [4 3; 2 1];
3 E = [1 1; 1 1];
4
5 % --- Adición de matrices ---
6 % La adición se realiza elemento a elemento. Las matrices deben tener las mismas dimensiones.
7 adicion = B + E;
8 disp('Resultado de la adición (B + E):');
9 disp(adicion);
10
11 % --- Substracción de matrices ---
12 % La substracción se realiza elemento a elemento.
13 sustraccion = B - E;
14 disp('Resultado de la substracción (B - E):');
15 disp(sustraccion);
16
17 % --- Multiplicación de matrices ---
18 % La multiplicación matricial estándar (producto punto) se realiza con el operador *.
19 % La multiplicación de matrices no es conmutativa (B * E no es lo mismo que E * B).
20 multiplicacion_estandar = B * E;
21 disp('Resultado de la multiplicación estándar (B * E):');
22 disp(multiplicacion_estandar);
23
24 % NOTA ADICIONAL: Multiplicación elemento a elemento
25 % El operador .* realiza la multiplicación elemento a elemento (producto de Hadamard).
26 % Esto es diferente de la multiplicación matricial estándar.
27 multiplicacion_elemento_a_elemento = B .* E;
28 disp('Resultado de la multiplicación elemento a elemento (B .* E):');
29 disp(multiplicacion_elemento_a_elemento);

line: 29 col: 42 encoding: SYSTEM (UTF-8) eol: LF
Command History Workspace Command Window Documentation Variable Editor Editor
```


Polinomio de Taylor para función e^x

$$e^x \approx 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots = 1 + \sum_{i=1}^N \frac{x^i}{i!}$$

*% Función que aproxima el valor de la función exponencial $\exp(x)$ a través de su
% polinomio de Taylor centrado en $x=0$. Las **entradas** de la función son:
% **1) x**: el punto x donde aproximar $\exp(x)$
% **2) tol**: tolerancia o diferencia entre aproximaciones consecutivas
% Las **salidas** de la función son:
% **1) ex**: aproximación de $\exp(x)$
% **2) i**: número de términos necesarios para que la diferencia entre aproximaciones
% consecutivas sea inferior (en valor absoluto) a tol*

*% Función que aproxima el valor de la función exponencial $\exp(x)$ a través de su
% polinomio de Taylor centrado en $x=0$. Las **entradas** de la función son:
% **1) x**: el punto x donde aproximar $\exp(x)$
% **2) tol**: tolerancia o diferencia entre aproximaciones consecutivas
% Las **salidas** de la función son:
% **1) ex**: aproximación de $\exp(x)$
% **2) i**: número de términos necesarios para que la diferencia entre aproximaciones
% consecutivas sea inferior (en valor absoluto) a tol*

```
function [ex,i] = MyExponential(x,tol)
ex    = 1;
i      = 0; %Contador para número de términos en el sumatorio
diff   = 1; %Damos valor a variable diff que permita entrar en el while
while diff>tol
i      = i + 1; %Incrementamos el contador del número de términos en el sumatorio
ex0    = ex; %Valor de la aproximación actual
ex     = ex0 + x^i/factorial(i); %Añadimos un término a la aproximación
diff   = abs(ex-ex0); %Calculamos diferencia entre aproximaciones consecutivas
endwhile
```

Función exponencialAprox.m

Programa principal

exponencialAprox.m

```
1 % Función que aproxima el valor de la función exponencial exp(x) a través de su
2 % polinomio de Taylor centrado en x=0.
3 % Entradas de la función:
4 % 1) x: el punto donde se quiere aproximar exp(x)
5 % 2) tol: la tolerancia o diferencia entre aproximaciones consecutivas
6 % Salidas de la función:
7 % 1) ex: aproximación de exp(x)
8 % 2) i: número de términos necesarios para que la diferencia entre
9 %     aproximaciones consecutivas sea inferior a tol
10
11 function [ex, i] = exponencialAprox(x, tol)
12 % Se inicializa la primera aproximación con el término constante (n=0),
13 % que es  $x^0/0! = 1/1 = 1$ .
14 ex = 1;
15 ex0 = 0; % ex0 es la aproximación anterior. Se inicializa en 0 para que la primera
16 % diferencia sea mayor que la tolerancia.
17 i = 0; % El contador de términos empieza en 0
18 % Bucle while para añadir términos hasta que se cumpla la tolerancia.
19 % La diferencia se calcula en cada iteración.
20 while abs(ex - ex0) > tol
21     i = i + 1; % Incrementa el contador de términos.
22     ex0 = ex; % Almacena la aproximación actual como la anterior para la siguiente iteración.
23     % Se añade el siguiente término de la serie de Taylor: ( $x^n / n!$ )
24     % La función factorial() en Octave ya existe, por lo que no es necesario
25     % implementarla manualmente a menos que se quiera.
26     ex = ex + (x^i / factorial(i));
27 end
28 end
```

exponencialAprox.m

prueba.m

```
1 % Script
2 clc;
3 clear;
4
5 x = 2; % Punto donde se aproximará exp(x)
6 tol = 1e-6; % Tolerancia deseada
7
8 % --- Llamada a la función ---
9 [aprox_exponencial, num_terminos] = exponencialAprox(x, tol);
10
11 % --- Resultados ---
12 disp(['Aproximando exp(', num2str(x), ')']);
13 disp('-----');
14
15 valor_exacto = exp(x);
16 disp(['Valor exacto de exp(', num2str(x), '): ', num2str(valor_exacto)]);
17
18 % Resultados de la aproximación
19 disp(['Aproximación de la serie de Taylor: ', num2str(aprox_exponencial)]);
20 disp(['Número de términos usados: ', num2str(num_terminos)]);
21
22 % Cálculo del error absoluto
23 error_absoluto = abs(valor_exacto - aprox_exponencial);
24 disp(['Error absoluto: ', num2str(error_absoluto)]);
```

- Para recuperar los datos almacenados en un archivo podemos utilizar el comando **load** (MATLAB/Octave) o **read** (Scilab) que lee datos del archivo y lo almacena en una variable:

MATLAB, Octave:

```
>> load datos.dat
```

Scilab:

```
>> read('datos.dat',m,n)
```

Guardamos el contenido del archivo `datos.dat` existente en el directorio, en una variable llamada **datos** (en Scilab debe especificarse su dimensión).

- Si deseamos que la variable tenga distinto nombre:

```
>> A = load('matriz.dat')
```

```
>> A = read('matriz.dat',m,n)
```

Guardamos el contenido del archivo `matriz.dat` en la variable `A`.

- En MATLAB si tipeamos:

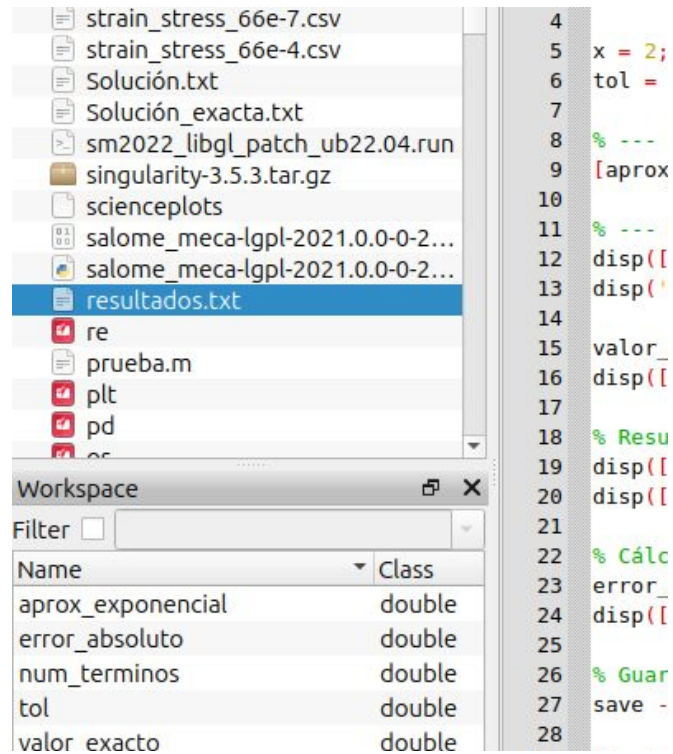
```
>> load
```

Recuperamos las variables definidas en la sesión anterior guardadas previamente con el comando `save`.

```
>> save datos.dat A -ascii
```

Guardamos la matriz `A` en el archivo `datos.dat` en formato `ascii`


```
1 % Script
2 clc;
3 clear;
4
5 x = 2;      % Punto donde se aproximará exp(x)
6 tol = 1e-6; % Tolerancia deseada
7
8 % --- Llamada a la función ---
9 [aprox_exponencial, num_terminos] = exponencialAprox(x, tol);
10
11 % --- Resultados ---
12 disp(['Aproximando exp(', num2str(x), ')']);
13 disp('-----');
14
15 valor_exacto = exp(x);
16 disp(['Valor exacto de exp(', num2str(x), '): ', num2str(valor_exacto)]);
17
18 % Resultados de la aproximación
19 disp(['Aproximación de la serie de Taylor: ', num2str(aprox_exponencial)]);
20 disp(['Número de términos usados: ', num2str(num_terminos)]);
21
22 % Cálculo del error absoluto
23 error_absoluto = abs(valor_exacto - aprox_exponencial);
24 disp(['Error absoluto: ', num2str(error_absoluto)]);
```



The screenshot shows the MATLAB interface. On the right, the 'Current Folder' browser displays a list of files, with 'resultados.txt' selected. Below it, the 'Workspace' browser shows a table of variables:

Name	Class
aprox_exponencial	double
error_absoluto	double
num_terminos	double
tol	double
valor_exacto	double

On the far right, a portion of the script code is visible, showing lines 4 through 28, including variable assignments and display commands.

% Guarda los resultados en un archivo de texto
save -ascii resultados.txt aprox_exponencial num_terminos;

Gráficas

MATLAB/Octave tienen cuatro funciones básicas para crear gráficos 2D:

- **plot** crea un gráfico a partir de vectores y/o columnas de matrices con escalas lineales sobre ambos ejes (idem en Scilab).
- **semilogx** Idem con escala lineal en el eje de ordenadas y logarítmica en el eje de abscisas.
- **semilogy** Idem con escala lineal en el eje de abscisas y logarítmica en el eje de ordenadas.
- **loglog** Idem con escala logarítmica en ambos ejes.

Gráficas

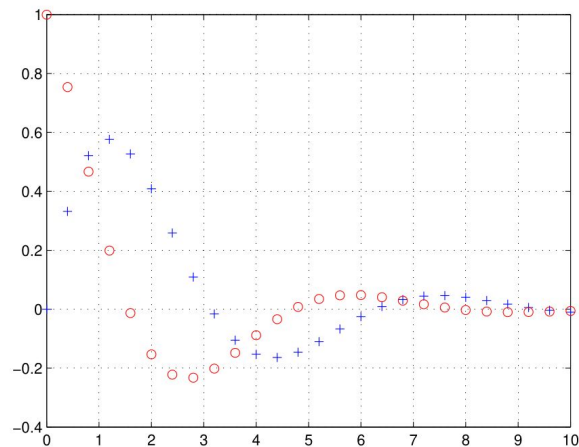
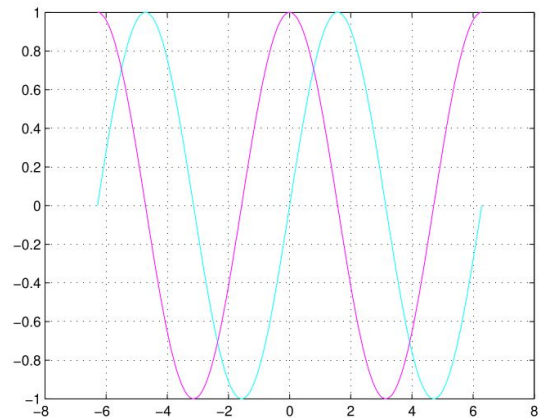
- Algunos colores, estilos y marcadores usuales del *plot* son:

Color	Símbolo
Azul	b
Verde	g
Rojo	r
Cyan	c
Magenta	m
Amarillo	y
Negro	k

Tipo de línea	Símbolo
Continua	-
A trazos	-
Punteada	:
A trazos y puntos	-.

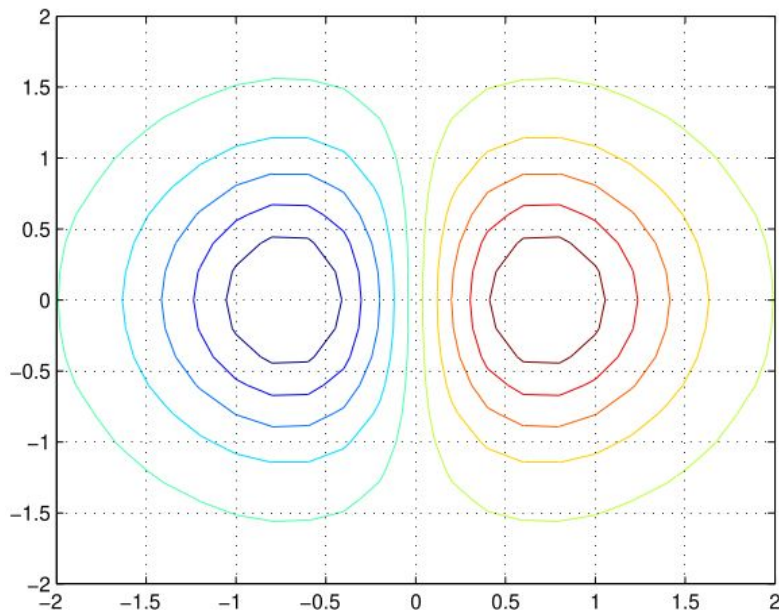
Tipo de marca	Símbolo
Puntos	.
Círculos	o
Cruces	x
Sumas	+
Asteriscos	*

```
plot(x,y1,'b+',x,y2,'ro'), grid on
```



Gráficas 2D

La función **contour(X,Y,Z,nivel)** sirve para graficar *curvas de nivel* de un campo escalar $z = f(x, y)$:



```
>> x = -2:0.2:2;  
>> y = x;  
>> [X,Y] = meshgrid(x,y); (uso meshgrid para hacer el "mallado")  
>> Z = X.*exp(-X.^2-Y.^2); (defino Z como matriz)  
>> contour(X,Y,Z,10), grid on (hago 10 curvas de nivel)
```

- También podemos utilizarlo para graficar funciones implícitas $f(x, y) = 0$ en 2D, por ejemplo, una parábola y una elipse:

$$\begin{cases} x^2 - 2x - y + \frac{1}{2} = 0 \\ x^2 + 4y^2 = 4 \end{cases}$$

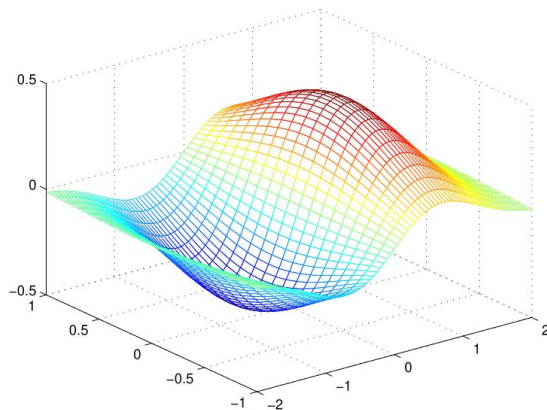
Gráficas 3D

$$z = f(x, y)$$

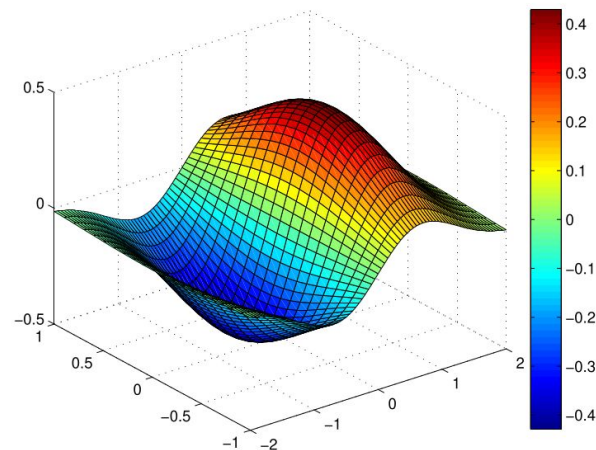
$$[a, b] \times [c, d]$$

```
>> x = -2:0.05:2;  
>> y = -1:0.1:1;  
>> [X,Y] = meshgrid(x,y);  
>> Z = X.*exp(-X.^2-Y.^2);
```

`mesh(X,Y,Z)`



`surf(X,Y,Z)`



- Operaciones vectoriales y matriciales
- Escribir una función definida en archivo
- Definir una función en línea o anónima
- Escribir un script que llame a la función
- Gráficas y sus propiedades



4. (a) Escriba una función llamada `cuad` que calcule las raíces de la ecuación cuadrática $ax^2 + bx + c = 0$ con $a, b, c \in \mathbb{R}, a \neq 0$, recibiendo como argumentos de entrada los coeficientes y devolviendo como salida las raíces:

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a},$$

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}.$$

```
function [x1, x2] = cuad(a, b, c)
    % Calcula el discriminante
    delta = b^2 - 4*a*c;

    % Calcula las raíces usando la fórmula
    cuadrática
    x1 = (-b + sqrt(delta)) / (2*a);
    x2 = (-b - sqrt(delta)) / (2*a);
end
```



4. (a) Escriba una función llamada `cuad` que calcule las raíces de la ecuación cuadrática $ax^2 + bx + c = 0$ con $a, b, c \in \mathbb{R}, a \neq 0$, recibiendo como argumentos de entrada los coeficientes y devolviendo como salida las raíces:

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a},$$

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}.$$

```
function [x1, x2] = cuad(a, b, c)
    % Calcula el discriminante
    delta = b^2 - 4*a*c;

    % Calcula las raíces usando la fórmula
    cuadrática
    x1 = (-b + sqrt(delta)) / (2*a);
    x2 = (-b - sqrt(delta)) / (2*a);
end
```



- (b) Modifique dicha función para que se imprima por pantalla indicando si las raíces son reales, reales iguales (en cuyo caso solo imprima una) o complejas conjugadas.

```
if delta > 0
```

```
elseif delta == 0
```

6. (a) Escriba la función `multi` que reciba como argumentos dos funciones y un vector de abscisas, y devuelva como resultado un vector con la multiplicación de ambas funciones evaluadas en dichas abscisas.
- (b) Calcule el producto de f_1 y f_2 del Ej. 7 en el vector $x = [-1, 0, 1]$ usando `multi`.

```
function resultado = multi(f1, f2, abscisas)
    % Evalúa la primera función en las abscisas
    y1 = f1(abscisas);

    % Evalúa la segunda función en las abscisas
    y2 = f2(abscisas);

    % Multiplica elemento a elemento los resultados y
    devuelve un vector
    resultado = y1 .* y2;
end
```

```
% Definición de las funciones f1 y f2
f1 = @(x) ...;
f2 = @(x) ....;
```

```
% Vector de abscisas
x = [-1, 0, 1];
```

```
% Llamada a la función multi para calcular el producto
producto = multi(f1, f2, x);

% Muestra el resultado
disp('Resultado del producto de las funciones f1(x)y
f2(x) en el vector x:');
disp(producto);
```

7. (a) Escriba archivos para evaluar cada una de las siguientes funciones:

- $f_1(x) = x^3 - x - 1$

- $f_3(x) = \cos(2x) - \sin(x)$

- $f_2(x) = e^{-x} - x$

- $f_4(x) = x \ln(|x|)$

(b) Grafíquelas en el intervalo $[-3, 3]$, todas en la misma gráfica y con distintos colores.

(c) Repita el item (b) pero graficando cada función en distintas gráficas de la misma figura usando el comando `subplot`.

```
x = -3:0.01:3;
```

```
f1 = x.^3 - x - 1;
```

```
f2 = exp(-x) - x;
```

```
f3 = cos(2*x) - sin(x);
```

```
f4 = x .* log(abs(x));
```

```
% Se corrige el valor en x=0 para  
evitar un 'log de cero'
```

```
f4(find(x==0)) = 0;
```

```
figure;  
  
% Gráfica de f1  
subplot(2, 2, 1);  
plot(x, f1, 'r');  
title('f1(x)');  
grid on;  
  
% Gráfica de f2  
subplot(2, 2, 2);  
plot(x, f2, 'g');  
title('f2(x)');  
grid on;  
  
% Gráfica de f3  
subplot(2, 2, 3);  
plot(x, f3, 'b');  
title('f3(x)');  
grid on;  
  
% Gráfica de f4  
subplot(2, 2, 4);  
plot(x, f4, 'm');  
title('f4(x)');  
grid on;
```

